

Real-world (rebuild a Project 1 feature, LCEL-style) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `from project_1 import extract_raw, ExtractedData` names your real Project 1 structured-extraction function and Pydantic model.

Basic Version

Approach 1 — print and eyeball

```
# lcel_vs_raw_sdk_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from project_1 import extract_raw, ExtractedData

prompt = ChatPromptTemplate.from_template("Extract structured data from: {text}")
lcel_chain = prompt | ChatOpenAI().with_structured_output(ExtractedData)

test_inputs = ["input one text", "input two text", "input three text"]

for text in test_inputs:
    raw_result = extract_raw(text)
    lcel_result = lcel_chain.invoke({"text": text})
    print("Input:", text)
    print("  raw: ", raw_result)
    print("  lcel:", lcel_result)
```

This shows both results side by side. It leaves the comparing to your own eyes, which doesn't scale past 3 inputs and is easy to get wrong — see Intermediate.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — actually assert the match

```
# lcel_vs_raw_sdk_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from project_1 import extract_raw, ExtractedData

def build_lcel_chain(model_name: str = "gpt-4o-mini"):
    prompt = ChatPromptTemplate.from_template("Extract structured data from: {text}")
    return prompt | ChatOpenAI(model=model_name, temperature=0).with_structured_output(ExtractedData)

def compare(text: str, lcel_chain) -> bool:
    raw_result = extract_raw(text)
    lcel_result = lcel_chain.invoke({"text": text})
    match = raw_result == lcel_result
    print(f"{text!r} -> match: {match}")
    if not match:
        print(f"  raw: {raw_result}")
        print(f"  lcel: {lcel_result}")
    return match

def main() -> None:
    lcel_chain = build_lcel_chain()
    test_inputs = ["input one text", "input two text", "input three text"]
    results = [compare(text, lcel_chain) for text in test_inputs]
    print(f"\n{sum(results)}/{len(results)} matched")

if __name__ == "__main__":
    main()
```

Expected output:

```
'input one text' -> match: True
'input two text' -> match: True
```

```
'input three text' -> match: True
```

```
3/3 matched
```

Difference from Basic: computing `match = raw_result == lcel_result` and reporting a final `N/M matched` count actually proves the comparison, instead of leaving it to a human reading printed text. Full type hints and a `build_lcel_chain()` function also make this reusable.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — timing both versions, per input and averaged

```
# lcel_vs_raw_sdk_practice.py
import time
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from project_1 import extract_raw, ExtractedData

def build_lcel_chain(model_name: str = "gpt-4o-mini"):
    prompt = ChatPromptTemplate.from_template("Extract structured data from: {text}")
    return prompt | ChatOpenAI(model=model_name, temperature=0).with_structured_output(ExtractedData)

def compare_timed(text: str, lcel_chain) -> tuple[bool, float, float]:
    start = time.perf_counter()
    raw_result = extract_raw(text)
    raw_seconds = time.perf_counter() - start

    start = time.perf_counter()
    lcel_result = lcel_chain.invoke({"text": text})
    lcel_seconds = time.perf_counter() - start

    match = raw_result == lcel_result
    print(f"{text!r} -> match: {match} (raw: {raw_seconds:.2f}s, lcel: {lcel_seconds:.2f}s)")
    return match, raw_seconds, lcel_seconds

def main() -> None:
    lcel_chain = build_lcel_chain()
    test_inputs = ["input one text", "input two text", "input three text"]

    rows = [compare_timed(text, lcel_chain) for text in test_inputs]
    matches = [r[0] for r in rows]
    raw_times = [r[1] for r in rows]
    lcel_times = [r[2] for r in rows]

    print(f"\n{sum(matches)}/{len(matches)} matched")
    print(f"Average raw: {sum(raw_times) / len(raw_times):.2f}s")
```

```
print(f"Average lcel: {sum(lcel_times) / len(lcel_times):.2f}s")

if __name__ == "__main__":
    main()
```

Expected output:

```
'input one text' -> match: True (raw: 0.81s, lcel: 0.87s)
'input two text' -> match: True (raw: 0.79s, lcel: 0.85s)
'input three text' -> match: True (raw: 0.83s, lcel: 0.89s)

3/3 matched
Average raw: 0.81s
Average lcel: 0.87s
```

The LCEL version is a little slower here, consistently — a small, close-to-negligible overhead from the extra layer, not a meaningful performance problem for this task.

Approach 2 — a reusable comparison table, with mismatches and timings saved for review

```
# lcel_vs_raw_sdk_practice.py
from dataclasses import dataclass
import time
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from project_1 import extract_raw, ExtractedData

@dataclass
class ComparisonRow:
    input_text: str
    raw_result: ExtractedData
    lcel_result: ExtractedData
    matched: bool
    raw_seconds: float
    lcel_seconds: float

def build_lcel_chain(model_name: str = "gpt-4o-mini"):
    prompt = ChatPromptTemplate.from_template("Extract structured data from: {text}")
    return prompt | ChatOpenAI(model=model_name, temperature=0).with_structured_output(
```

```

def run_comparison(test_inputs: list[str]) -> list[ComparisonRow]:
    lcel_chain = build_lcel_chain()
    rows = []
    for text in test_inputs:
        start = time.perf_counter()
        raw_result = extract_raw(text)
        raw_seconds = time.perf_counter() - start

        start = time.perf_counter()
        lcel_result = lcel_chain.invoke({"text": text})
        lcel_seconds = time.perf_counter() - start

        rows.append(ComparisonRow(
            text, raw_result, lcel_result, raw_result == lcel_result, raw_seconds
        ))
    return rows

def main() -> None:
    test_inputs = ["input one text", "input two text", "input three text"]
    rows = run_comparison(test_inputs)

    mismatches = [row for row in rows if not row.matched]
    avg_raw = sum(r.raw_seconds for r in rows) / len(rows)
    avg_lcel = sum(r.lcel_seconds for r in rows) / len(rows)

    print(f"{len(rows) - len(mismatches)}/{len(rows)} matched")
    print(f"Average raw: {avg_raw:.2f}s, average lcel: {avg_lcel:.2f}s "
          f"({(avg_lcel / avg_raw - 1) * 100:+.1f}% vs. raw)")
    for row in mismatches:
        print(f"MISMATCH on: {row.input_text}")
        print(f"  raw: {row.raw_result}")
        print(f"  lcel: {row.lcel_result}")

if __name__ == "__main__":
    main()

```

Expected output:

```

3/3 matched
Average raw: 0.81s, average lcel: 0.87s (+7.4% vs. raw)

```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate proves the two versions agree, and stops there. Approach 1 adds real timing to that same comparison, turning "LangChain has some overhead" into a specific, measured number. Approach 2 keeps

everything from Approach 1 but collects it into a `ComparisonRow` list, so this comparison — correctness *and* timing — can be saved, extended to more inputs, or turned into an actual regression check with almost no changes, instead of being a script you only ever read once.

Which one should you actually run? Intermediate's correctness check is the minimum this document's Build Task actually requires. Run Approach 1's timing once, for yourself, when you're first deciding whether LangChain's overhead is worth worrying about for a given task — for most single-call features, it isn't. Keep Approach 2's structured version around if you expect to re-run this comparison regularly (every time you touch the prompt, or upgrade a LangChain version) — the percentage-overhead line is exactly the number you'd want to notice creeping upward over time.